

FROM ZERO TO HERO

Building a Local RAG Pipeline

Private, Open-Source, and Python-Powered

Manish Dwibedy
PyDelhi

| The 2026 Local Stack

HIGH-PERFORMANCE OPEN-SOURCE TOOLS



Compute

NVIDIA GPU or Apple
M-Series Silicon.



Inference

Ollama serving GGUF
models locally.



Storage

Qdrant or ChromaDB for
vectors.

Manish Dwibedy

2012-2015 TCS @ Gurugram - TMobile & Vodafone Australia

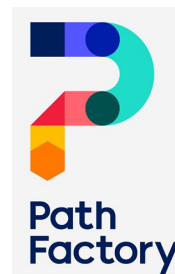
2015-2017 MS @ USC - Data Science

2018-2020 ML @ Cisco & ABB, California, US

2020-2022 ML @ PathFactory & GoodUnited, India,

2022 - Now Founder @ Ajitha Tech, Ahan Tech

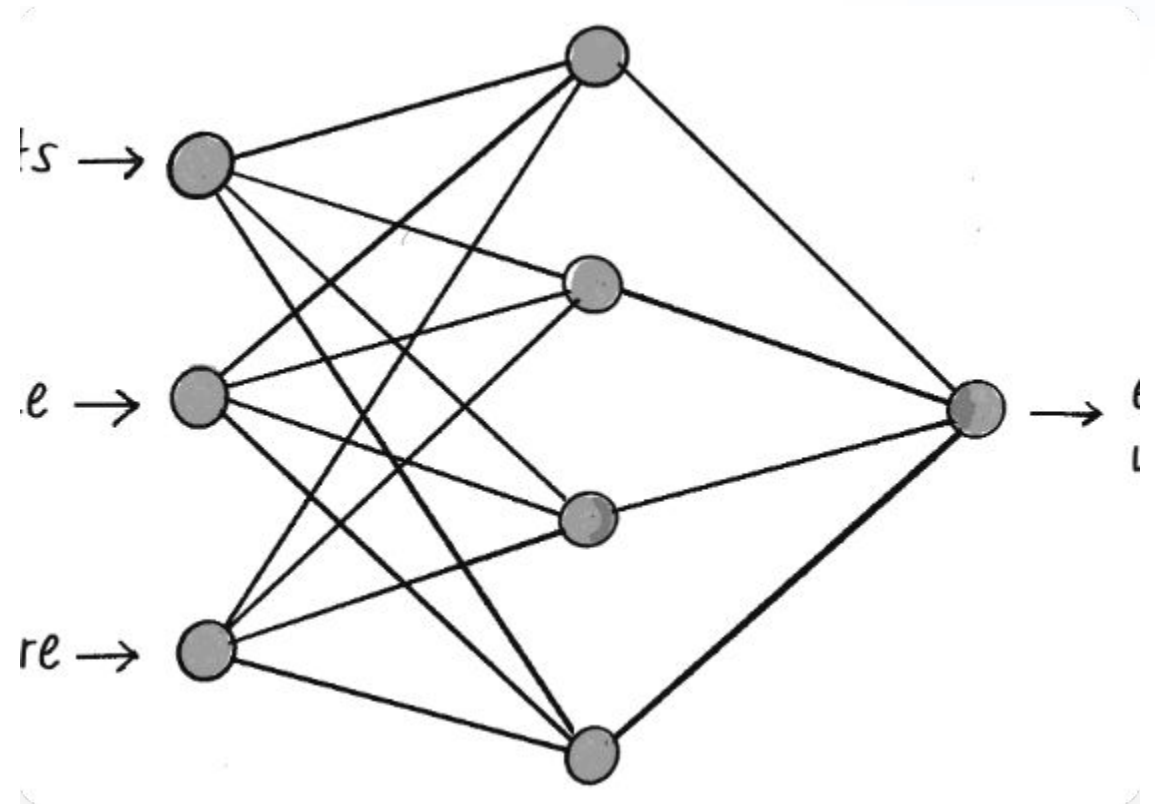
2025 - Now Products for Social Media, Fintech, Upcoming Author



| Architecture Overview

THE BIG PICTURE OF DATA FLOW

- Data Ingestion
- Smart Storage
- Semantic Retrieval
- Synthesis & Generation



| Advanced Data Ingestion

BEYOND SIMPLE PDF TEXT EXTRACTION



The Challenge

Tables, embedded images, and complex multi-column layouts break naive parsers.



The Solution

Use Unstructured.io or Docling to convert raw data into clean Markdown.

| The Art of Chunking

PART 1: BASIC STRUCTURAL SPLITTING

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=50,
    separators=["\n\n", "\n", " ", ""], # default style
)
```

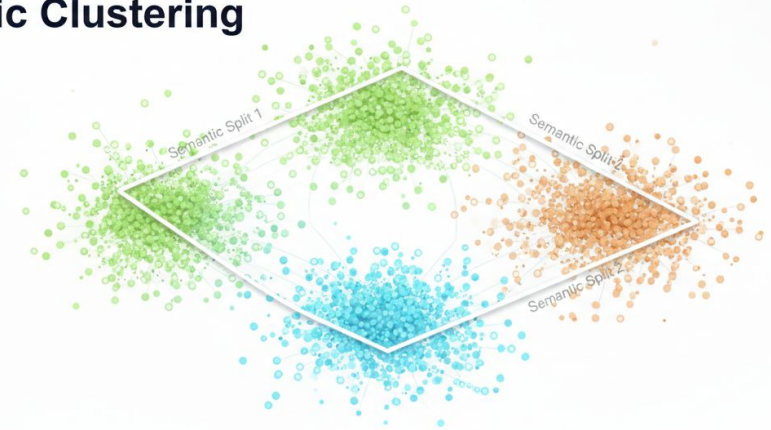
Fixed-size is quick; Recursive is context-aware.

| Advanced Chunking

PART 2: SEMANTIC & HIERARCHICAL

- 📖 **Semantic Chunking:** Split based on topic shifts via similarity.
- 👤 **Parent-Document:** Search small chunks, retrieve large context.

Semantic Clustering



| Choosing the Right Lens

EMBEDDING MODEL PERFORMANCE



Tip: Your search is only as good as your embedding model's vocabulary.

Vector Database Engine

Engine	Best For	Storage Type	Cost Model
ChromaDB	Prototyping & Simplicity	SQLite / Local Disk	Lowest Entry Cost: Free (Open Source)
FAISS	High-speed Search	In-memory	Zero Software Cost: Free library. High Hardware Cost (GPU/RAM)
Qdrant	Production Hybrid Search	High-performance Disk	Predictable Scaling: Free (Open Source)

| Vector Database Engine

ChromaDB: The Developer's "Easy Button"

Core Focus: Developer experience and rapid AI app iteration.

Architecture: SQLite-backed; lightweight and "embeddable" in Python.

Quantization: Basic **Scalar Quantization** (converts decimals to smaller integers).

Best For:

Getting a RAG app running in under 10 minutes.

Small-to-mid datasets (<1M vectors) on local machines.

| Vector Database Engine

FAISS: The "Bare Metal" Library

Core Focus: Maximum search throughput and high-dimensional math.

Architecture: Optimized C++ library (not a full DB) with **GPU acceleration**.

Quantization: Advanced **Product Quantization (PQ)**; fits billions of vectors into RAM.

Best For:

Research-grade performance where you manage the storage yourself.

Heavy-duty offline processing of static datasets.

| Vector Database Engine

Qdrant: The "Production-Hardened" Engine

Core Focus: Maximum search throughput and high-dimensional math.

Architecture: Optimized C++ library (not a full DB) with **GPU acceleration**.

Quantization: Advanced **Product Quantization (PQ)**; fits billions of vectors into RAM.

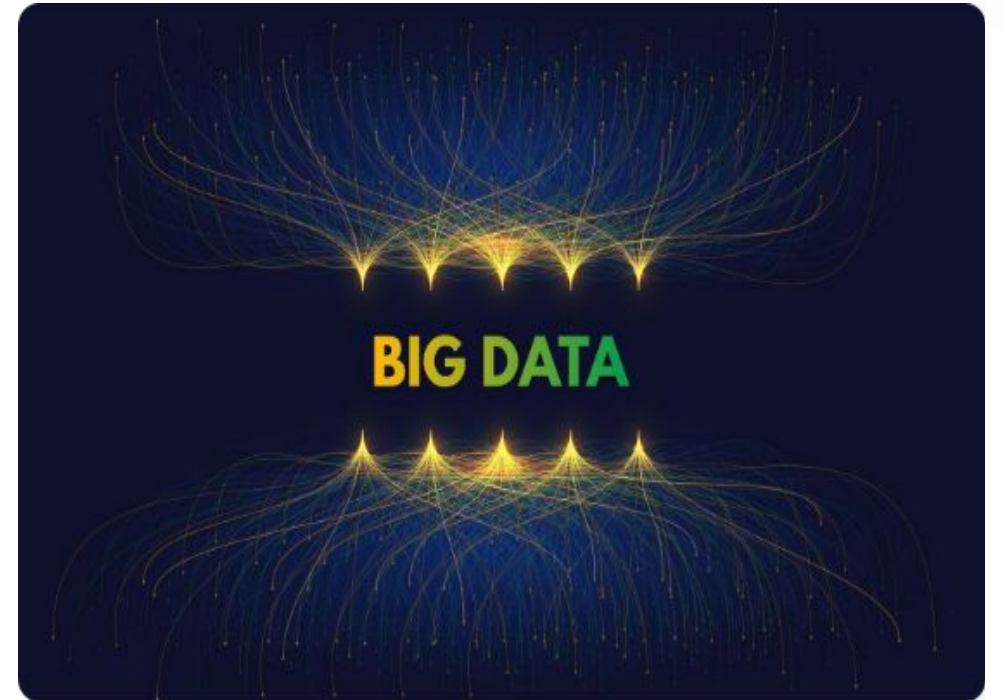
Best For:

Research-grade performance where you manage the storage yourself.

Heavy-duty offline processing of static datasets.

| Creating the Vector Store

```
from langchain_community.embeddings import  
HuggingFaceEmbeddings  
  
# BGE Small EN v1.5 embedding model  
embeddings = HuggingFaceEmbeddings(  
    model_name="BAAI/bge-small-en-v1.5",  
    model_kwargs={"device": "cuda"},  
    encode_kwargs={"normalize_embeddings": True}  
)
```



| Local LLM with Ollama

THE EASIEST WAY TO SERVE LOCAL MODELS



One Command

```
ollama pull llama3:8b
```



LangChain Integration

```
ChatOllama(model="llama3")
```

| Quantization Demystified

70%

VRAM Savings

Performance vs Memory

4-bit quantization (Q4_K_M) offers massive memory savings with **less than 1% accuracy loss**.

| Hybrid Search Strategy

COMBINING VECTOR AND KEYWORD POWER

Semantic (Vector)

Captures meaning and synonyms. Good for general concepts.

Keyword (BM25)

Exact matches for Product IDs, names, or technical codes.

Combined via **Reciprocal Rank Fusion (RRF)**

Hybrid Implementation

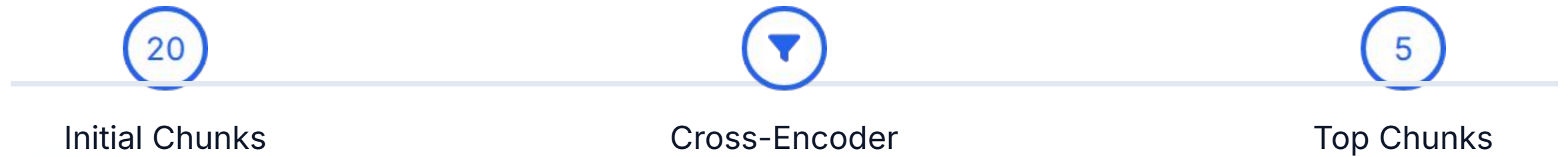
```
from langchain.retrievers import EnsembleRetriever
from langchain_community.retrievers import BM25Retriever
from langchain_community.vectorstores import FAISS

bm25_retriever = BM25Retriever.from_documents(docs)
faiss_store = FAISS.from_documents(docs, embeddings)
faiss_retriever = faiss_store.as_retriever(search_kwargs={"k": 2})

ensemble = EnsembleRetriever(
    retrievers=[bm25_retriever, faiss_retriever],
    weights=[0.7, 0.3], # adjust weights as needed
)
```

| Power of Reranking

REDUCING NOISE AND HALLUCINATIONS



Using **BAAI/bge-reranker-base** for final validation.

| Local Reranker Setup

```
from langchain.retrievers import ContextualCompressionRetriever
from langchain_community.document_compressors import FlashrankRerank

compressor = FlashrankRerank()

compression_retriever = ContextualCompressionRetriever(
    base_compressor=compressor,
    base_retriever=retriever
)
```

Drastically improves "Signal-to-Noise" ratio in prompts.

| Query Transformation

PRE-RETRIEVAL OPTIMIZATION



Multi-Query

Generate 3 variants of the query to broaden context search.



HyDE

Generate a "fake" answer and use it to search for real documents.

| Context Management

THE "LOST IN THE MIDDLE" PROBLEM

- ❗ LLMs focus on the **start and end** of prompts.
- ✅ Place vital chunks at the **top** of the context.
- ✂ Trim irrelevant data to prevent token fatigue.

| System Prompts

BUILDING STRICT GUARDRAILS

"Use ONLY the provided context.

If the answer is not in context, say 'I don't know'.

Cite sources using [Source X]."

| The Final Chain

```
from langchain.chains import create_retrieval_chain

rag_chain = create_retrieval_chain(retriever, document_chain)

response = rag_chain.invoke({
    "input": "How do I scale local RAG?"
})
```

| Pipeline Evaluation

HOW DO WE KNOW IT WORKS?



RAGAS

The industry standard framework for RAG evaluation.



LLM-as-a-Judge

Use a strong local LLM (Llama 3) to score the output of another.

RAGAS Metrics

Metric	Goal
Faithfulness	Is the answer derived solely from context?
Relevance	Does it actually answer the user's question?
Context Recall	Did we find all relevant information?

Reranking – The "Quality Filter" for RAG

The Problem: Vector search is fast but "fuzzy." It finds similarities, not always **exact relevance**.

The Solution: A **Reranker** (Cross-Encoder) acts as a second-pass filter. It scores the top 10–50 results from your Vector DB to find the perfect match.

The Impact: Drastically improves **answer accuracy** and reduces **LLM hallucinations** by providing higher-quality context.

Top Solutions:

- **Cohere Rerank:** Easiest to deploy via API.
- **BGE-Reranker:** Top-tier open-source performance.
- **FlashRank:** Optimized for ultra-low latency.

| Key Takeaways

Start Simple

Naive RAG works for 60% of cases. Don't over-engineer early.

Optimize Later

Add Reranking and Hybrid search as accuracy needs grow.

Stay Local

Your data is your most valuable asset. Keep it private.

Questions?

LinkedIn - <https://www.linkedin.com/in/manishdwibedy/>

Portfolio - <https://manishd.in/>

Final Tip: "The quality of your RAG is 80% data cleaning and 20% model selection."

